Sign in to Medium with Google

Morgan Daniel
morganjdaniel@gmail.com

Morgan Daniel
morg270804@gmail.com

2 more accounts

Source True · Follow publication

WebGPU performan

15 min read · Nov 22, 2023

Open in app ↗ Sign up Sign in

Medium Search

> *Discover the truth about WebGPU performance in this eye-opening article. Is it living up to the expectations? We delve into the data and real-world experiences to uncover the answers and explore what lies ahead for this revolutionary technology. Brace yourself for surprising insights and a glimpse into the future of the web!*

Why WebGPU? 🤔

In recent times, the popularity of brand-new WebGPU has been increasing. The WebGPU API enables web developers to use the underlying system's GPU to deal with high-performance computations and draw complex graphics that can be rendered in the browser.

In 2011, when WebGL first appeared, it brought a breakthrough in graphics for the web. WebGL is a JavaScript port of OpenGL ES 2.0, allowing web pages to pass

rendering computations directly to the device's GPU. Later in 2016, WebGL2 was introduced, which provides an interface for the OpenGL ES 3.0 rendering context.

However, WebGL still has some limitations as it is an implementation of OpenGL. New generations of native GPU APIs came, including Microsoft's Direct3D 12, Apple's Metal, and The Khronos Group's Vulkan, and they provide new GPU abilities that were not planned to be implemented in OpenGL and therefore in WebGL. WebGL is based on the use case of drawing graphics and rendering them to a canvas. And it does not handle general-purpose GPU computations very well. GPU computations are becoming more and more important in many different areas, for example, machine learning and artificial intelligence.

WebGPU is the successor to WebGL, it provides better compatibility with modern GPUs, gives access to advanced GPU features, and is designed with first-class support for GPU computations.

What will we test with WebGPU? 🧪

For performance measures, we will use particle system animation similar to what we implemented before in JavaScript and WASM performance comparisons.

The particles will have collision checks with each other and with container boundaries.

We will expose some metric values to the global scope and use Puppeteer for auto-tests to measure FPS.

A project will be implemented in Typescript, as it has nice type annotations for WebGPU, and it will be much easier to use the new web API with it.

The already-implemented project is presented on GitHub, and the demo is deployed on this page.

Particles Implementation 👨‍💻

As WebGPU is still an experimental technology, before we start to write code, we need to add additional configuration for Typescript. Add the `@webgpu/types` dependency to `package.json` and add the following line to tsconfig.json:

```
{
  "compilerOptions": {
```

```
    ...
    "typeRoots": [ "./node_modules/@webgpu/types", "./node_modules/@types"]
  }
  ...
}
```

To make it easier to understand, we will break the code into the following blocks:

- WebGPU initialization
- WebGPU buffer creation
- Particles initialization pipeline
- Particles update pipeline
- Particles render pipeline
- Particles computing and rendering passes

WebGPU initialization

Let's start to add some lines of code to the newly created `index.ts` file.

```
/* Declare constants with values that will be used later */
const canvasSizePx = 500;
const particleSizePx = 5;
const particleSize = particleSizePx / canvasSizePx;
const defaultParticleAmount = 3000;
const particleStateSize = 4 /* x, y, speedX, speedY */;

/* Get number of particles from url params. */
const urlParams = new URLSearchParams(window.location.search);
const rawParticles = urlParams.get("particles");
const particleAmount = rawParticles
  ? Number(rawParticles)
  : defaultParticleAmount;

/* Will display actual FPS on UI */
const fpsElement = document.getElementById('fps') as HTMLParagraphElement;

/* Expose API on the Windows interface for automation */
interface Window {
  __FPS__?: number;
}

/* Entry point */
```

```
async function initWebGPU() {
  /* Check WebGPU support */
  if (!navigator.gpu) {
    throw new Error("WebGPU not supported on this browser.");
  }

  /* Adapter identifies an implementation of WebGPU on the system */
  const adapter = await navigator.gpu.requestAdapter();
  if (!adapter) {
    throw new Error("No appropriate GPU adapter found.");
  }

  /* Device is the logical instantiation of an adapter */
  const device = await adapter.requestDevice();

  /* Setup WebGPU canvas context */
  const canvas = document.getElementById("canvas") as HTMLCanvasElement;

  canvas.width = canvasSizePx;
  canvas.height = canvasSizePx;

  const context = canvas.getContext("webgpu") as GPUCanvasContext;

  const canvasFormat = navigator.gpu.getPreferredCanvasFormat();

  context.configure({
    device: device,
    format: canvasFormat,
  });

  ...
}

initWebGPU();
```

Most WebGPU API calls are asynchronous, so `initWebGPU` is an asynchronous function as well.

Note that `webgpu` canvas context is optional, and it's not needed if WebGPU is used only for computations.

WebGPU buffer creation

Below, in `initWebGPU`, let's provide data to a `GPUBuffer`.

```
/* Rectangle with coordinates for particle */
const vertices = new Float32Array([
  -1, -1, 1, -1, 1, 1,
```

```
  -1, -1, 1, 1, -1, 1,
]);

/* Vertex buffer */
const vertexBuffer = device.createBuffer({
  label: "Particle vertices",
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});

/* Write vertex buffer to gpu */
device.queue.writeBuffer(vertexBuffer, 0, vertices);

/* Global variables for calculations in shaders */
const uniformArray = new Float32Array([
  particleAmount,
  particleSize,
/* Seeds for random function in shaders */
  Math.random(),
  Math.random(),
  Math.random(),
  Math.random(),
]);

/* Uniform buffer */
const uniformBuffer = device.createBuffer({
  label: "Particle Uniforms",
  size: 32,
  usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
});

/* Write uniform buffer to gpu */
device.queue.writeBuffer(uniformBuffer, 0, uniformArray);

/* Holds x/y coordinates and x/y speed for each particle */
const particleStateArray = new Float32Array(
  particleStateSize * particleAmount,
);

/* Storage buffer list */
const particleStateBuffers = [
  device.createBuffer({
    label: "Particle State A",
    size: particleStateArray.byteLength,
    usage: GPUBufferUsage.STORAGE, GPUBufferUsage.COPY_DST,
  }),
  device.createBuffer({
    label: "Particle State B",
    size: particleStateArray.byteLength,
    usage: GPUBufferUsage.STORAGE, GPUBufferUsage.COPY_DST,
  }),
];
```

```
/* Write storage buffer to gpu with initial index */
device.queue.writeBuffer(particleStateBuffers[ 0 ], 0, particleStateArray);
```

The `vertices` variable consists of twelve items, as each couple of values represents a point in `x` and `y` coordinates, which means there are six points; thus, a rectangle is created by the combination of two triangles. For values, normalized device coordinates are why values are in the range from `-1` to `1`.

For particle state, we created two storage buffers because later we will use a 'ping-pong' pattern for that when one buffer is only for reading and another—for writing—and then buffers are swapped on each frame.

Below, we are defining additional layouts and binding groups.

```
/* Will be used in render pipeline */
const vertexBufferLayout: GPUVertexBufferLayout = {
  arrayStride: /* x */ 4 + /* y */ 4,
  stepMode: "vertex",
  attributes: [
    {
      format: "float32x2",
      offset: 0,
      shaderLocation: 0,
    },
  ],
};

/* Define resources bound and accessibility in shaders */
const bindGroupLayout = device.createBindGroupLayout({
  label: "Bind Group Layout",
  entries: [
    {
      binding: 0,
      visibility:
        GPUShaderStage.VERTEX |
        GPUShaderStage.FRAGMENT |
        GPUShaderStage.COMPUTE,
      buffer: {},
    },
    {
      binding: 1,
      visibility:
        GPUShaderStage.VERTEX | GPUShaderStage.COMPUTE,
      buffer: { type: "read-only-storage" },
    },
```

```
      {
        binding: 2,
        visibility: GPUShaderStage.COMPUTE,
        buffer: { type: "storage" },
      },
    ],
  });

  /* Define bind group layout for pipeline */
  const pipelineLayout = device.createPipelineLayout({
    label: "Particle Pipeline Layout",
    bindGroupLayouts: [ bindGroupLayout ],
  });

  /* Create two binding groups for ping-pong buffer pattern */
  const bindGroups = [
    device.createBindGroup({
      label: "Bind Group A",
      layout: bindGroupLayout,
      entries: [
        {
          binding: 0,
          resource: { buffer: uniformBuffer },
        },
        {
          binding: 1,
          resource: { buffer: particleStateBuffers[0] },
        },
        {
          binding: 2,
          resource: { buffer: particleStateBuffers[1] },
        },
      ],
    }),
    device.createBindGroup({
      label: "Bind Group B",
      layout: bindGroupLayout,
      entries: [
        {
          binding: 0,
          resource: { buffer: uniformBuffer },
        },
        {
          binding: 1,
          resource: { buffer: particleStateBuffers[1] },
        },
        {
          binding: 2,
          resource: { buffer: particleStateBuffers[0] },
        },
      ],
    }),
  ];
```

Particles initialization pipeline

Now it is time to write the first WebGPU shader for the initialization pipeline.

WebGPU shader language (WGSL) has a different syntax than GLSL, but it's designed to support two types of GPU commands:

- a dispatch command for the compute pipeline.
- a draw command for the render pipeline.

Let's add the `initStateShader` string variable and put WGSL here:

```
/* Declare global constants */
const stateOffset: u32 = 4;
const minSpeed: f32 = 0.004;
const maxSpeed: f32 = 0.012;

/* Store random seed as private variable */
var<private> rand_seed : vec2<f32>;

/* Initialize random seed */
fn init_random(index : u32, seed : vec4<f32>) {
  rand_seed = seed.xz;
  rand_seed = fract(rand_seed * cos(35.456+f32(index) * seed.yw));
  rand_seed = fract(rand_seed * cos(41.235+f32(index) * seed.xw));
}

/* GPU random function which passed on random seed mutation */
fn random() -> f32 {
  rand_seed.x = fract(
    cos(dot(rand_seed, vec2<f32>(23.14077926, 232.61690225)) * 136.8168,
  );

  rand_seed.y = fract(
    cos(dot(rand_seed, vec2<f32>(54.47856553, 345.84153136)) * 534.7645,
  );

  return rand_seed.y;
}

/* Returns random x and y values */
fn randomPosition(particleSize: f32) -> f32 {
  return particleSize + random() * (2 - 2 * particleSize) - 1;
}
```

```
/* Returns random speed for x and y coordinates */
fn randomSpeed() -> f32 {
  let speed = minSpeed + random() * (maxSpeed - minSpeed);

  if (random() > 0.5) {
    return speed;
  } else {
    return -speed;
  }
}

/* Custom structure for uniforms */
struct ParticleUniforms {
  particleAmount: f32,
  particleSize: f32,
  seed: vec4f,
};

/* Binds to uniform buffer */
@group(0) @binding(0) var<uniform> particleUniforms : ParticleUniforms;
/* Binds to writable state buffer */
@group(0) @binding(2) var<storage, read_write> particleStateOut: array<f32>;

@compute /* Compute shader entry point */
@workgroup_size(16) /* Amount of invocations per shaded workgroup */
/* Only 'x' dimension will be changed for invocation ID */
fn computeMain(@builtin(global_invocation_id) particle: vec3u) {
  let particleAmount = u32(particleUniforms.particleAmount);

  /* Aboart computation when invocation ID exceeds particle amount */
  if (particle.x >= particleAmount) {
    return;
  }

  /* Define index according to invocation ID and state size */
  let index = particle.x * stateOffset;

  init_random(index, particleUniforms.seed);

  /* Set initial x */
  particleStateOut[index] = randomPosition(particleUniforms.particleSize);
  /* Set initial y */
  particleStateOut[index + 1] = randomPosition(particleUniforms.particleSize);
  /* Set initial speed x */
  particleStateOut[index + 2] = randomSpeed();
  /* Set initial speed y */
  particleStateOut[index + 3] = randomSpeed();
}
```

In the shader above, we have the attribute, `@workgroup_size` , which specifies the x, y, and z dimensions of the workgroup grid for the compute shader. The workgroup is a set of invocations that concurrently execute a compute shader stage entry point and share access to shader variables in the workgroup address space.

Get Leo's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Continue to set the pipeline for the compute shader in the `initWebGPU` function:

```
/* Create shader module from initStateShader string */
const initStateShaderModule = device.createShaderModule({
  label: "Init Particles State",
  code: initStateShader,
});

/* Create compute pipeline for initStateShader */
const initStatePipeline = device.createComputePipeline({
  label: "Init Particles State Pipeline",
  layout: pipelineLayout,
  compute: {
    module: initStateShaderModule,
    entryPoint: "computeMain",
  },
});
```

Particles update pipeline

Shade code: in `updateStateShader`, the string will be like this:

```
/* Size of state for one particle in state buffer */
const stateOffset: u32 = 4;

/* Calculates distance between two points */
fn distance(x1: f32, y1: f32, x2: f32, y2: f32) -> f32 {
    let dx = x1 - x2;
```

```
    let dy = y1 - y2;
    return sqrt(dx * dx + dy * dy);
}

/* Custom structure for uniforms */
struct ParticleUniforms {
  particleAmount: f32,
  particleSize: f32,
  seed: vec4f,
};

/* Binds to uniform buffer */
@group(0) @binding(0) var<uniform> particleUniforms : ParticleUniforms;
/* Binds to read-only state buffer */
@group(0) @binding(1) var<storage, read> particleStateIn: array<f32>;
/* Binds to writable state buffer */
@group(0) @binding(2) var<storage, read_write> particleStateOut: array<f32>;

@compute /* Compute shader entry point */
@workgroup_size(16, 16) /* Amount of invocations per shaded workgroup */
/* 'x' and 'y' dimensions are used for invocation IDs */
fn computeMain(@builtin(global_invocation_id) particle: vec3u) {
  let particleAmount = u32(particleUniforms.particleAmount);

  /* Aboart computation when invocation ID exceeds particle amount */
  if particle.x >= particleAmount
    || particle.y >= particleAmount {
    return;
  }

  /* Define index according to x invocation ID and state size */
  let xIndex = particle.x * stateOffset;
  let yIndex = xIndex + 1;

  let speedXIndex = xIndex + 2;
  let speedYIndex = xIndex + 3;

  /* Current particle position */
  let x = particleStateIn[xIndex];
  let y = particleStateIn[yIndex];

  var speedX: f32;
  var speedY: f32;

  /* Check collision with container boundies only once */
  if particle.y == 0 {
    /* Current particle speed */
    speedX = particleStateIn[speedXIndex];
    speedY = particleStateIn[speedYIndex];

    let halfSize = particleUniforms.particleSize / 2;

    /* Check collision with wall by x axis */
```

```
      if (x >= 1 - halfSize && speedX > 0)
        || (x <= -1 + halfSize && speedX < 0) {
        /* Change speed x to opposite value */
        speedX = -speedX;
      }

      /* Check collision with wall by y axis */
      if (y >= 1 - halfSize && speedY > 0)
        || (y <= -1 + halfSize && speedY < 0) {
        /* Change speed y to opposite value */
        speedY = -speedY;
      }

      /* Assign speed to writable state */
      particleStateOut[speedXIndex] = speedX;
      particleStateOut[speedYIndex] = speedY;
    }

    /* Take latest speed from writable state */
    speedX = particleStateOut[speedXIndex];
    speedY = particleStateOut[speedYIndex];

    /* Calculate index of other particle */
    let nextIndex = particle.y * stateOffset;

    /* Check if it's not same particle */
    if nextIndex != xIndex {
      /* Other particle position */
      let nextX = particleStateIn[nextIndex];
      let nextY = particleStateIn[nextIndex + 1];

      let dist = distance(x, y, nextX, nextY);

      /* Check collision with other particle by distance */
      if dist <= particleUniforms.particleSize && dist > 0 {
        /* Change speed x and y to opposite values */
        particleStateOut[speedXIndex] = -speedX;
        particleStateOut[speedYIndex] = -speedY;
      }
    }

    /* Update particle position when all collisions are checked */
    if particle.y == particleAmount - 1 {
      particleStateOut[xIndex] = x + speedX;
      particleStateOut[yIndex] = y + speedY;
    }
  }
```

And create a shader module and compute a pipeline for updating the particle's state accordingly:

```
/* Create shader module from updateStateShader string */
const updateStateShaderModule = device.createShaderModule({
  label: "Update Particles State",
  code: updateStateShader,
});

/* Create compute pipeline for updateStateShader */
const updateStatePipeline = device.createComputePipeline({
  label: "Update Particles State Pipeline",
  layout: pipelineLayout,
  compute: {
    module: updateStateShaderModule,
    entryPoint: "computeMain",
  },
});
```

Particles render pipeline

A render shader principle is similar to GLSL, and it consists of vertex and fragment shaders. And if you are familiar with WebGL, it would not be a big deal to write a GPU program in WGSL.

```
/* State size */
const stateOffset: u32 = 4;
/* Particle drawing constants */
const radius: f32 = 1.0;
const lineWidth: f32 = 0.4;
const colorMultiplier: f32 = 0.66;

/* Inter-stage variables */
struct VertexOutput {
/* Updated vertex clip position */
  @builtin(position) position: vec4f,
/* Original position from vertex buffer */
  @location(0) particlePosition: vec2f,
};

/* Structure for uniforms */
struct ParticleUniforms {
  particleAmount: f32,
  particleSize: f32,
  seed: vec4f,
};
```

```
/* Binds to uniform buffer */
@group(0) @binding(0) var<uniform> particleUniforms : ParticleUniforms;
/* Binds to read-only state buffer */
@group(0) @binding(1) var<storage, read> particleStateArray: array<f32>;

@vertex /* Vertex shader entry point */
fn vertexMain(
  /* Current vertex within the current API-level draw command */
  @builtin(instance_index) instance: u32,
  /* Position from vertex buffer */
  @location(0) position: vec2f,
) -> VertexOutput {
  /* Paticle index in state according to instance index and state size */
  let index = instance * stateOffset;

  /* Inter-stage instance */
  var output: VertexOutput;

  /* Multiply by scaling and apply position change from state to state */
  output.position = vec4f(
    position.x * particleUniforms.particleSize
      + particleStateArray[index], /* x */
    position.y * particleUniforms.particleSize
      + particleStateArray[index + 1], /* y */
    0, /* depth */
    1, /* perspective divisor */
  );

  /* Store original vertex position */
  output.particlePosition = vec2f(position.x, position.y);

  /* Pass output to fragment shader */
  return output;
}

@fragment /* Fragment shader entry point */
fn fragmentMain(
  @location(0) particlePosition: vec2f
) -> @location(0) vec4f {
  /* Distance to center of clip */
  let distance = length(vec2f).
    particlePosition.x, particlePosition.y,
  ));

  /* Caclulate fragment for circle as rgb */
  let circle = vec3f(
    step(radius, line width, distance) - step(radius, distance)
  );

  /* Fragment color in format rgba */
  return vec4f(circle * colorMultiplier, 1.0);
}
```

By utilizing an inter-stage structure, we can define which fields to pass from the vertex to the fragment shader.

In the vertex shader, we multiply `position` by scale and add displacement from the storage. `particlePosition` stores the vertex position without changes for the fragment shader.

The fragment shader draws a circle shape of a given radius with a defined line thickness and color in RGBA format.

The render pipeline configuration accordingly:

```
/* Shader module from renderShader string */
const renderShaderModule = device.createShaderModule({
  label: "Render Particle Shader",
  code: renderShader,
});

/* Render pipeline for renderShader */
const renderPipeline = device.createRenderPipeline({
  label: "Render Particles Pipeline",
  layout: pipelineLayout,
  vertex: {
    module: renderShaderModule,
    entryPoint: "vertexMain",
    buffers: [ vertexBufferLayout ],
  },
  fragment: {
    module: renderShaderModule,
    entryPoint: "fragmentMain",
    targets: [
      {
        format: canvasFormat,
      },
    ],
  },
  primitive: {
    /* Instructs to draw triangles from vertecies */
    topology: 'triangle-list',
  },
});
```

Particles computing and rendering passes

The final part of the code for computing and rendering passes execution:

```
async function initWebGPU() {
  ...

  /* Tracks current update step */
  let step = 0;

  /* Command encoder for particles initialisation */
  const encoder = device.createCommandEncoder();

  /* Compute pass for particles initialisation */
  const initStatePass = encoder.beginComputePass();
  /* Set pipeline to compute pass */
  initStatePass.setPipeline(initStatePipeline);
  /* Set bind group to compute pass by initial step */
  initStatePass.setBindGroup(0, bindGroups[step]);

  /* Invocations per shader workgroup, limited by GPU */
  const WORKGROUP_SIZE = 16;
  /* Calculate required amount of workgroup */
  const workgroupCount = Math.ceil(particleAmount / WORKGROUP_SIZE);

  /* Apply workgroups to initialisation pass for x dimension */
  initStatePass.dispatchWorkgroups(workgroupCount);
  /* End initialisation pass */
  initStatePass.end();

  /* Finish encoder and submit command bufffer to gpu queue */
  const commandBuffer = encoder.finish();
  device.queue.submit([ commandBuffer ]);

  /* Variables for fps clculating */
  let fps = 0;
  let fpsCounter = 0;
  let fpsTimestamp = 0;
  const fpsCount = 10;
  const second = 1000;

  /* Update callback on each frame */
  function update(time: number) {
    /* Increment step */
    step++;

    /* Command encoder for update and render passes */
    const encoder = device.createCommandEncoder();

    /* Compute pass for particles update */
    const updateStatePass = encoder.beginComputePass();
    /* Set pipeline to update state pass */
```

```
    updateStatePass.setPipeline(updateStatePipeline);
    /* Toggle bing group each frame for compute pass */
    updateStatePass.setBindGroup(0, bindGroups[step % 2]);
    /* Apply workgroups to init pass for x dimension */
    updateStatePass.dispatchWorkgroups(workgroupCount, workgroupCount);
    /* End update pass */
    updateStatePass.end();

    /* Render particles pass */
    const renderPass = encoder.beginRenderPass({
      /* Output to when executing this render pass */
      colorAttachments: [
        {
          view: context.getCurrentTexture().createView(),
          /* Clear value for this attachment */
          loadOp: "clear",
          /* Initial viev color */
          clearValue: { r: 0, g: 0, b: 0, a: 1 },
          /* Stores the resulting value to this attachment */
          storeOp: "store",
        },
      ],
    });

    /* Set pipeline to render pass */
    renderPass.setPipeline(renderPipeline);
    /* Set vertex buffer to render pass */
    renderPass.setVertexBuffer(0, vertexBuffer);
    /* Toggle bing group each frame for render pass */
    renderPass.setBindGroup(0, bindGroups[ step % 2 ]);
    /* Draw primitives from vertices and instance it 'particleAmount' times */
    renderPass.draw(vertices.length / 2, particleAmount);
    /* End render pass */
    renderPass.end();

    /* Finish encoder and submit command bufffer to gpu queue */
    const commandBuffer = encoder.finish();
    device.queue.submit([ commandBuffer ]);

    /* Calculate FPS each 'fpsCount' time */
    if (step % fpsCount === 0) {
      const delta = time - fpsTimestamp;
      fps = (second * fpsCount) / delta;
      window.__FPS__ = fps;
      fpsElement.innerText = "fps: " + fps.toPrecision(4);

      fpsTimestamp = time;
    }

    /* Request update loop */
    window.requestAnimationFrame(update);
  }
```

```
    /* Request initial update */
    window.requestAnimationFrame(update);
  }

  /* Entry point call */
  initWebGPU();
```

You may have noticed that the bind groups are toggled each time for an update and render pipelines. It refers to the *ping-pong buffer pattern*. The idea is that we use two copies of the state, and on each frame, we read from one copy of the state and write the result to the other. So we ensure that we always perform the next update using the results from the previous frame.

Results 📈

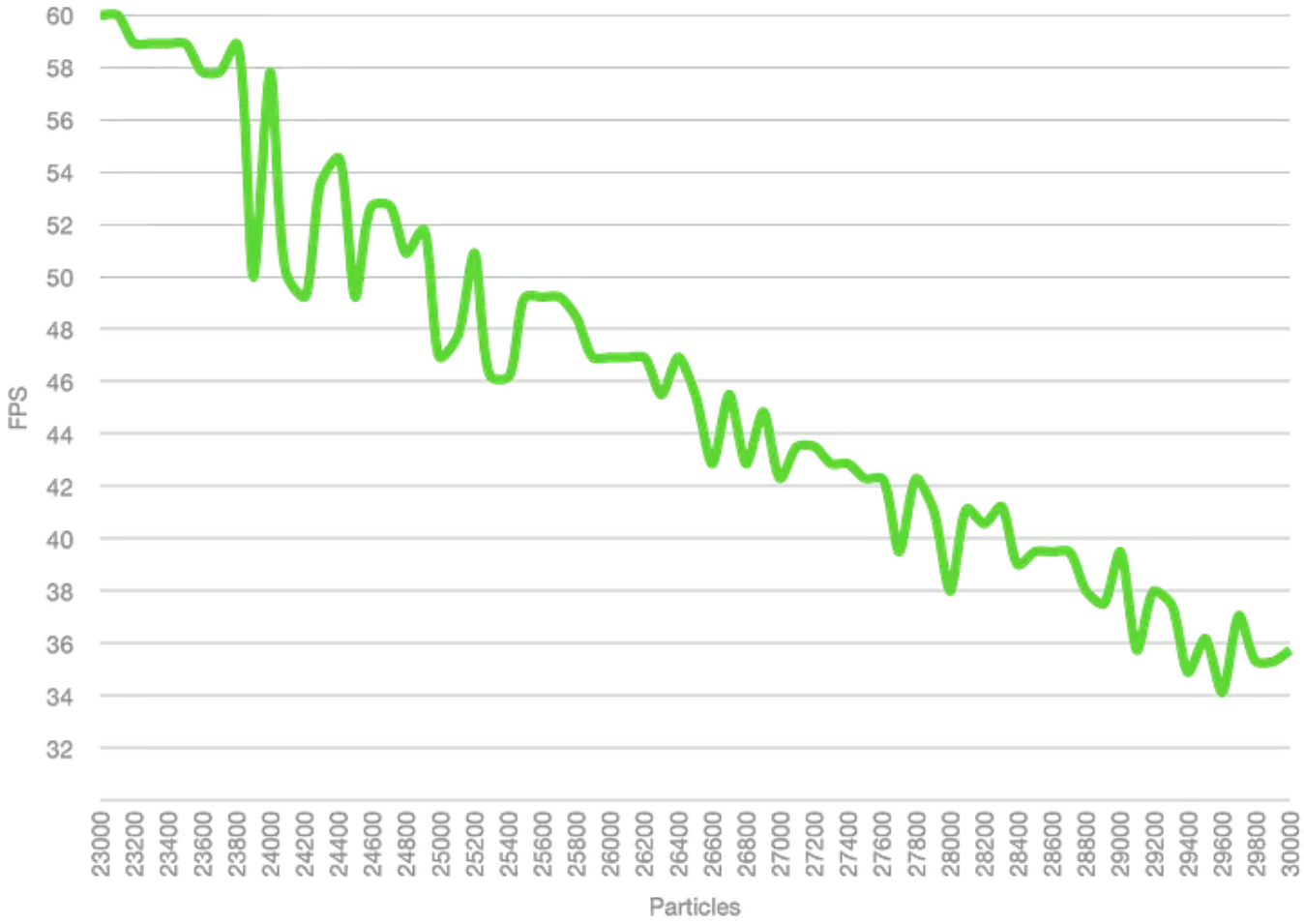
The results were captured from a 16-inch MacBook Pro 2019 with the following characteristics:

- Processor: 2.3 GHz 8-Core Intel Core i9
- Graphics: AMD Radeon Pro 5500M 4 GB
- Memory: 16 GB, 2667 MHz DDR4
- macOS Ventura Version 13.6

The project has been run with Puppet using Chromium with version 115.0.5790.102.

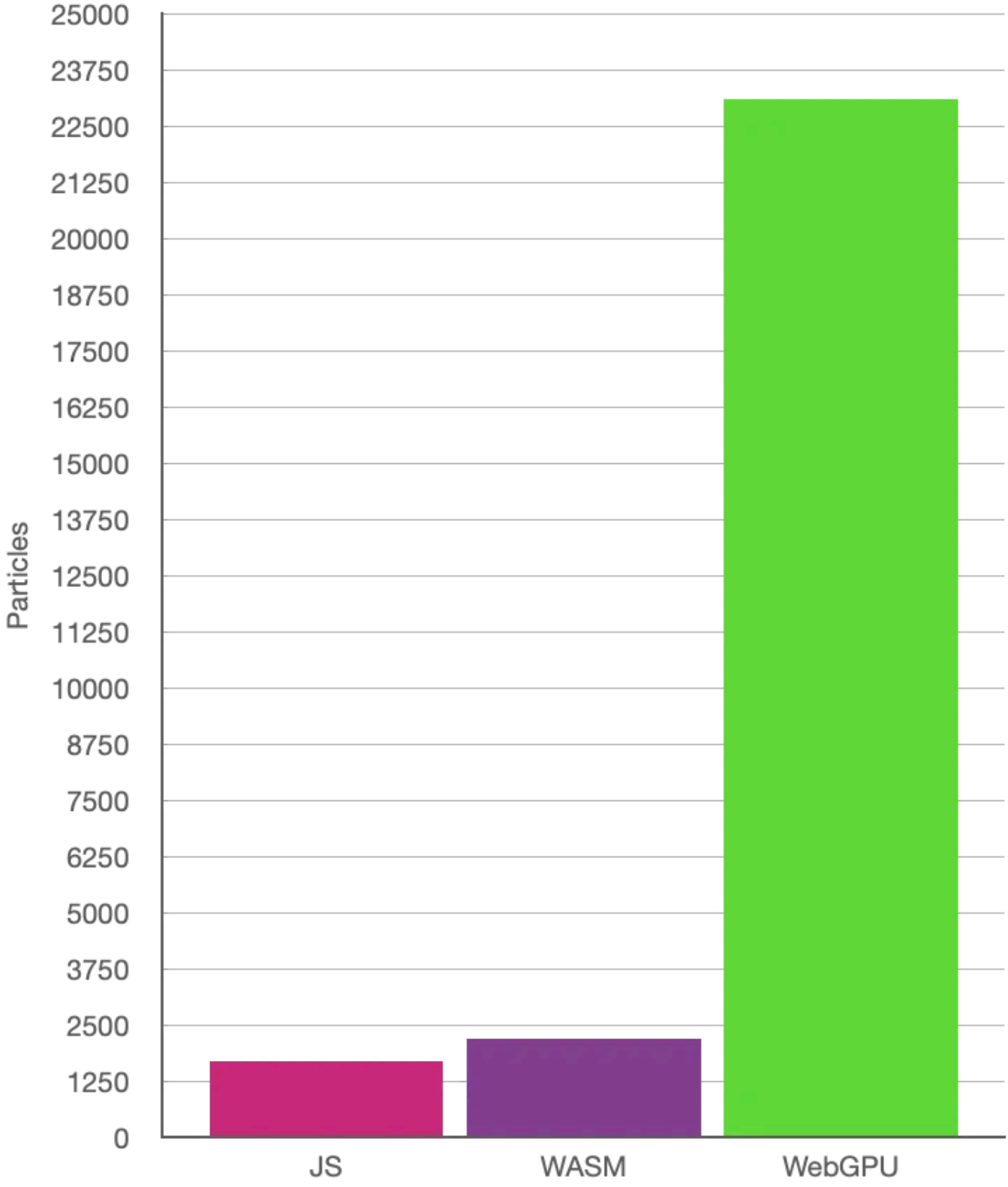
We skip CPU consumption this time as computation is mostly done on GPUs.

Let's focus on the most interesting measures for this particular case — FPS depends on the number of particles. Surprisingly, the refresh rate remains constant for more than 23k particles. Even for 30k particles, there are still more than 30 frames per second.



FPS dependency on a number of particles.

Comparing the maximum number of particles at which FPS remains 60 for native JavaScript, WebAssembly, and WebGPU It's around 10 times the advantage of WebGPU over JS, and WASM:



Max particles for which FPS remains 60.

Network and loading time ⏱️

Let's apply network throttling and simulate a slow 3G internet connection.

Finally, check the network tab in DevTools and look at the HTTP requests and their processing time.

Name	Status	Type	Initiator	Size	Time	Waterfall
webgpu/?particles=3000	200	document	Other	492 B	2.03 s	
index.css	200	stylesheet	webgpu/?particles=3000:5	458 B	2.00 s	
index.js	200	script	webgpu/?particles=3000:12	3.4 kB	2.10 s	

3 requests | 4.4 kB transferred | 14.7 kB resources | Finish: 4.13 s | DOMContentLoaded: 4.13 s | Load: 4.13 s

Network statistics on slow 3G.

According to the network, we see only a few resources required for a simple web application. There are no additional requests as WGSL shaders are inlined in the `index.ts` file. With included shaders index.js is still pretty small, only 3.4kb. The total loading time for a slow 3G connection is 4.13 seconds.

Summary 💬

In this situation, WebGPU outperforms Javascript and WebAssembly without compromising loading time. Also, it is worth mentioning that the CPU stays free and it could perform other tasks in parallel to GPU.

WebGPU was first published in 2021, and its still experimental API is supported only by the most recent browser versions and available via the feature flag (Google Chrome started to support WebGPU by default in May 2023). Additionally, it takes time to learn the new WGSL.

However, some popular libraries like TensorFlow.js and Babylon.js already use WebGPU acceleration.

Before using WebGPU, browser support and WGSL willingness should be considered.

References 📚

- W3C — WebGPU.
- W3C — WebGPU Shading Language.
- MDN — WebGPU API.
- GitHub — Performance comparison between JavaScript, WebAssembly, and WebGPU.
- Your first WebGPU app.

Web Development | JavaScript | Wgsl | Webgpu

Follow

Published in Source True

4 followers · Last published Nov 22, 2023

Software development experiments and code tricks.

Follow

Written by Leo

46 followers · 42 following

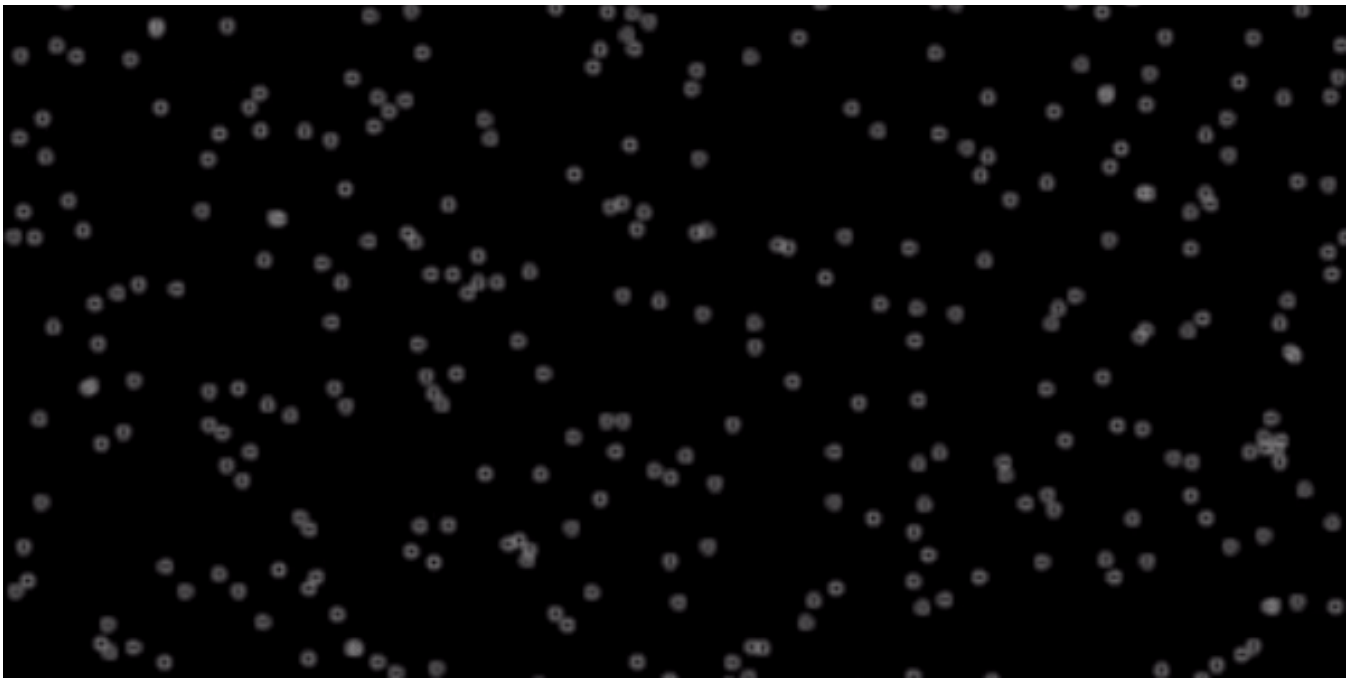
JavaScript/TypeScript developer. In past ActionScript 3.0. https://stesel.netlify.app/

No responses yet

Write a response

What are your thoughts?

More from Leo and Source True



In Source True by Leo

JavaScript vs WebAssembly performance for Canvas particle system

In 2017 WebAssembly was declared for all magor browsers and since that time it generate a lot of noize, owing to be able to run code on a...

Jan 12, 2023 94 1

In Source True by Leo

Why is Entity Component System (ECS) so awesome for game development?

During game development, we sometimes want to consider new design patterns. In this case, we can try out the ECS, which has been used more...

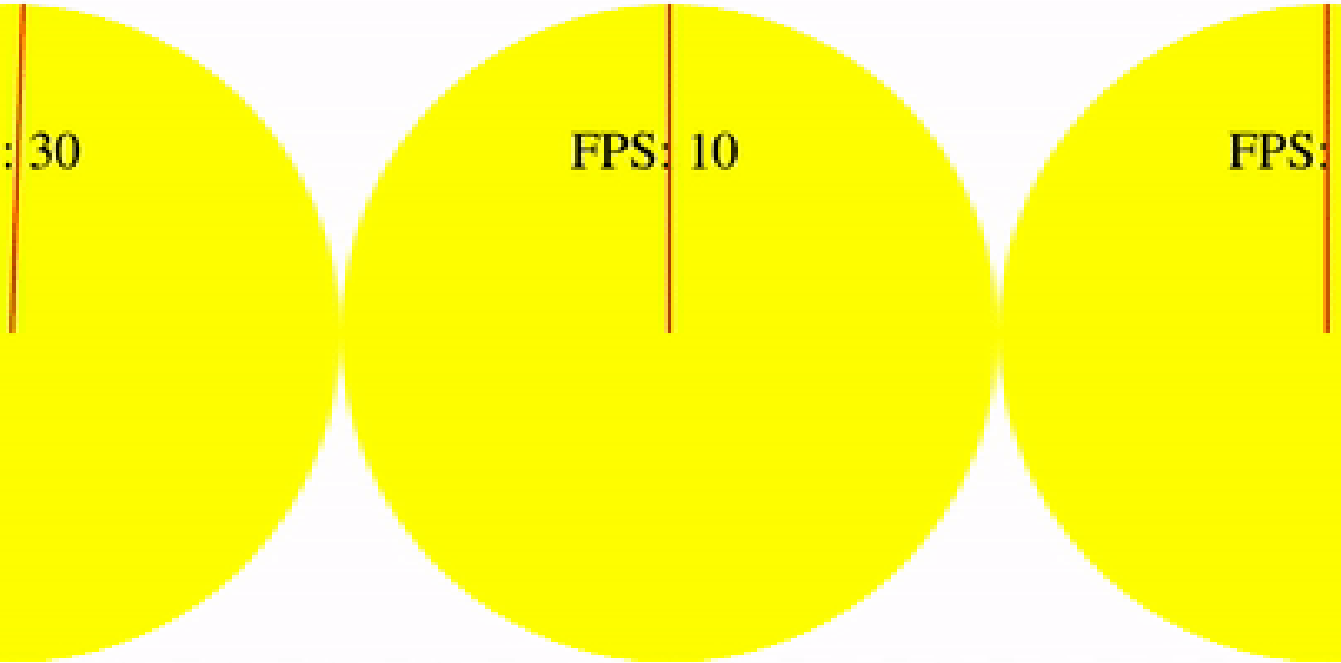
Dec 9, 2022 20

In Source True by Leo

String Vibration Physics Animation with React

Lets implement a physics of string vibration using React.

Jun 15, 2019 201 1



In Source True by Leo

Animate a React component with a configurable frame rate

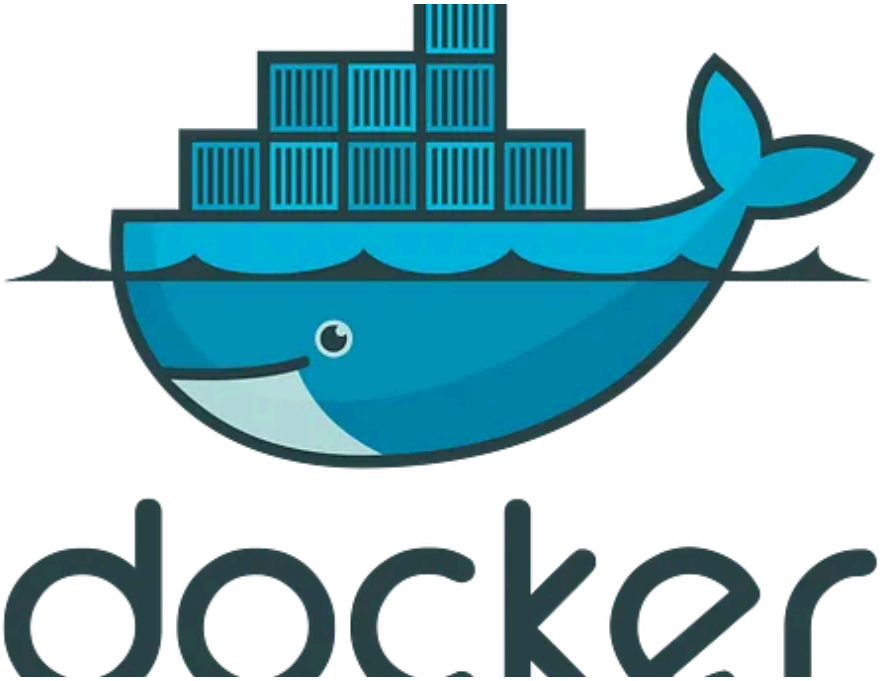
Developing a web application on React suddenly we want to add a fancy animation.

Feb 18, 2019 195 1

See all from Leo

See all from Source True

Recommended from Medium



Abhinav

Docker Is Dead—And It's About Time

Docker changed the game when it launched in 2013, making containers accessible and turning "Dockerize it" into a developer catchphrase.

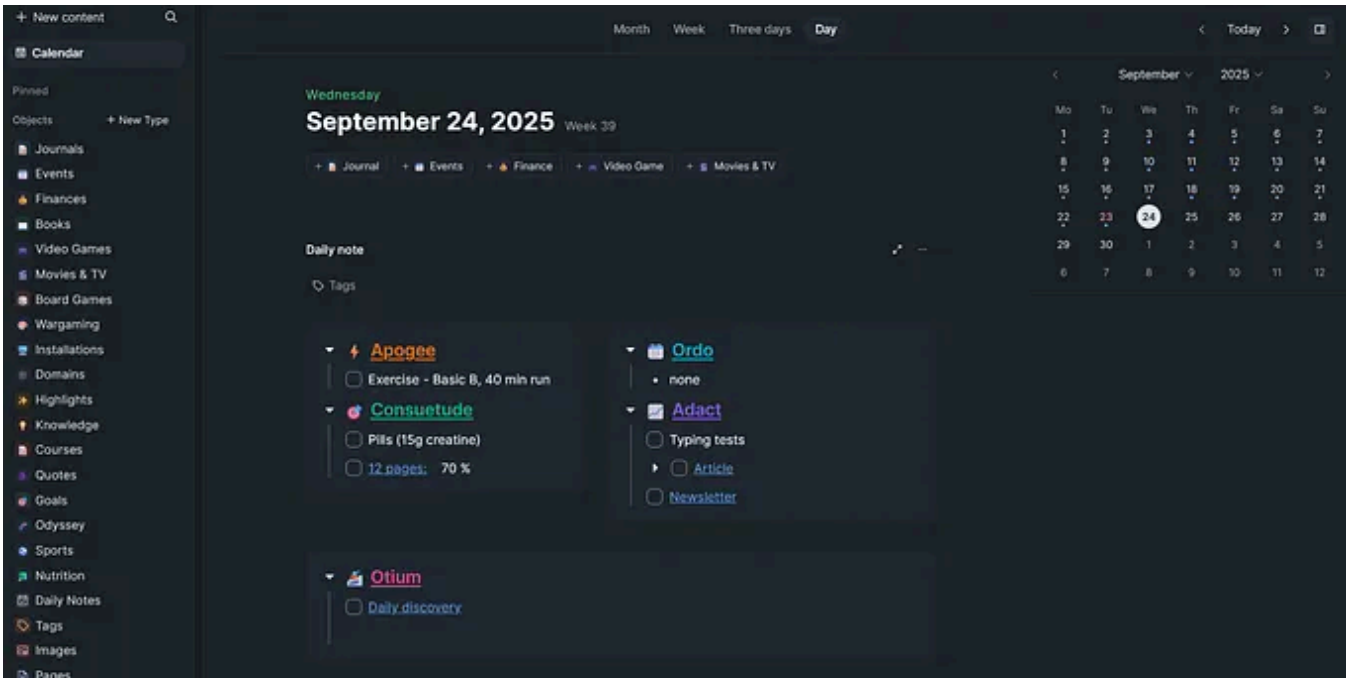
Jun 9 6.9K 192

In Long. Sweet. Valuable. by Ossai Chinedum

I'll Instantly Know You Used Chat Gpt If I See This

Trust me you're not as slick as you think

May 16 25K 1483



Tosny

7 Websites I Visit Every Day in 2025

If there is one thing I am addicted to, besides coffee, it is the internet.

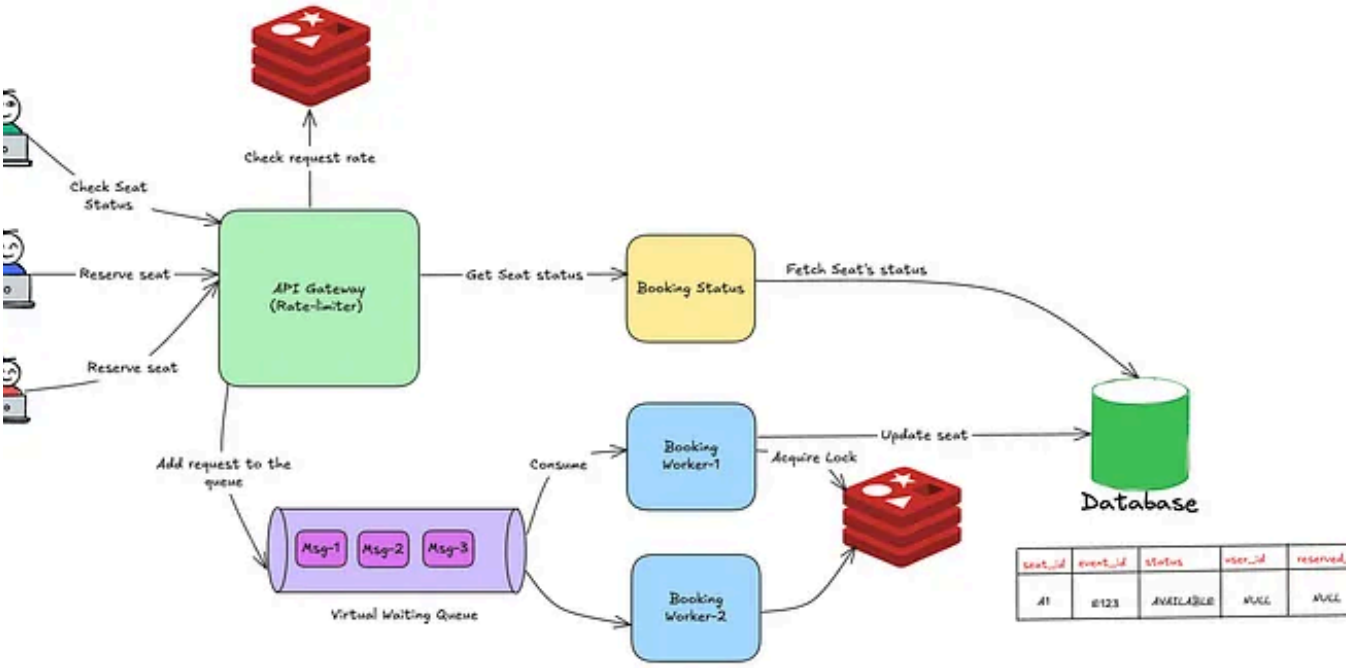
Sep 23 4.1K 166

Yash Batra

The 47-Line Code That Made One Developer $2 Million from AI

In late 2024, a solo indie developer pushed 47 lines of Python code to GitHub.

Jul 8 4.2K 130



In ITNEXT by Animesh Gaitonde

Solving Double Booking at Scale: System Design Patterns from Top Tech Companies

Learn how Airbnb, Ticketmaster, and booking platforms handle millions of concurrent reservations without conflicts

Oct 8 1K 20

Nadia Makarevich

React State Management in 2025: What You Actually Need

Learn how to manage state in modern React apps, what is remote, URL, local, and shared state, and when you need a state management library

Oct 2 202 10

See more recommendations